

ThermalML

Spacecraft Component Temperature Prediction & Overheat Anomaly
Detection Using Machine Learning

Project Report

Course: Fundamentals of AI and ML

Submission Type: Flipped Course Evaluation — Build Your Own Project

1. Introduction

Spacecraft components in low Earth orbit (LEO) are exposed to a continuously varying thermal environment shaped by direct solar radiation, Earth albedo, and Earth infrared (IR) emission, further modulated by orbital altitude, sun angle, and eclipse duration. Accurately predicting a component's temperature — and detecting when it risks exceeding a safe operating limit — is central to spacecraft thermal design. This project, built for the Fundamentals of AI and ML course, applies supervised machine learning (regression and classification) to this problem, using a physics-grounded synthetic dataset in place of proprietary satellite telemetry.

2. Problem Statement

Given a set of orbital and thermal-design parameters (altitude, sun angle, surface absorptivity/emissivity, surface area, eclipse fraction, albedo coefficient, and incident flux values), predict (a) the steady-state temperature of a spacecraft component in degrees Celsius, and (b) whether that component is at risk of overheating (temperature $\geq 85^{\circ}\text{C}$). The goal is to demonstrate that ML models trained on physically simulated data can approximate the results of traditional thermal-balance calculations quickly and accurately, offering a fast, data-driven complement to detailed thermal analysis during early-stage design iteration.

3. Functional Requirements

- **Module 1 — Data Generation & Preprocessing:** Generate a physics-based synthetic dataset from the radiative energy balance equation; validate schema and value ranges; clean invalid rows; scale features for modeling.
- **Module 2 — Model Training (Regression + Classification):** Train and compare Linear Regression and Random Forest regressors to predict component temperature; train a Random Forest classifier to flag overhear risk.
- **Module 3 — Evaluation & Visualization:** Compute RMSE/MAE/ R^2 for regression and Accuracy/Precision/Recall/F1 + confusion matrix for classification; generate feature importance and diagnostic plots.

Input/output structure: inputs are 10 numeric orbital/thermal features; outputs are a predicted temperature ($^{\circ}\text{C}$), a binary overhear-risk flag, and an overhear probability. Workflow: the user runs the CLI (*generate data* $\rightarrow$ *preprocess* $\rightarrow$ *train* $\rightarrow$ *evaluate* $\rightarrow$ *predict*), or imports the ThermalPredictor class directly for programmatic single/batch predictions.

4. Non-Functional Requirements

- **Performance:** Full pipeline (5,000-row dataset generation, training two regressors and one classifier) completes in under 10 seconds on a standard CPU.
- **Reliability:** Explicit schema validation (DataValidationError) and row-level cleaning guard against malformed or missing data before training.

- **Maintainability:** Configuration values are centralized in config.py; each pipeline stage is isolated in its own module with a single responsibility.
- **Usability:** A single CLI (main.py) with clear flags (--generate-data, --train, --predict, --all) runs the entire workflow without requiring users to read the source code.
- **Logging/Monitoring:** Every module logs key events (dataset stats, training progress, evaluation metrics) via a shared logger configuration.
- **Resource efficiency:** Models are persisted with joblib so trained artifacts can be reused for prediction without retraining.

5. System Architecture

The system is organized as a linear pipeline of independent modules coordinated by a CLI entry point, with trained models and generated data persisted to disk for reuse.

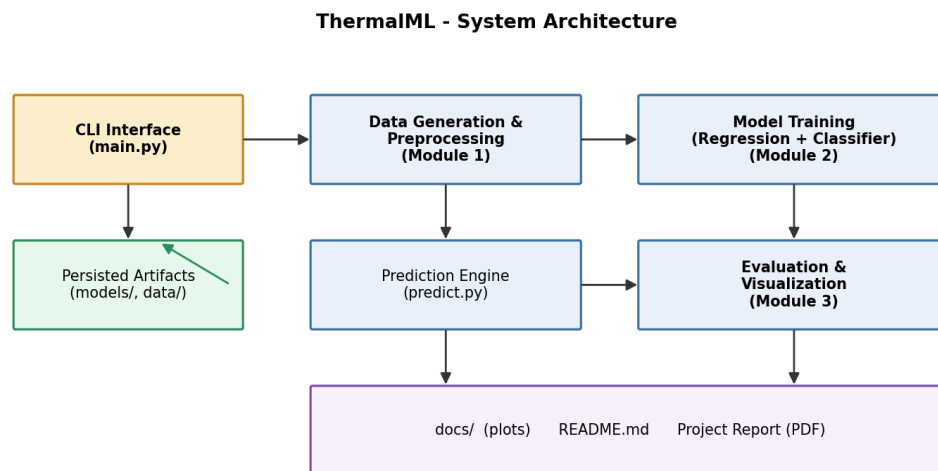


Figure 1: System architecture

5.1 Process Workflow

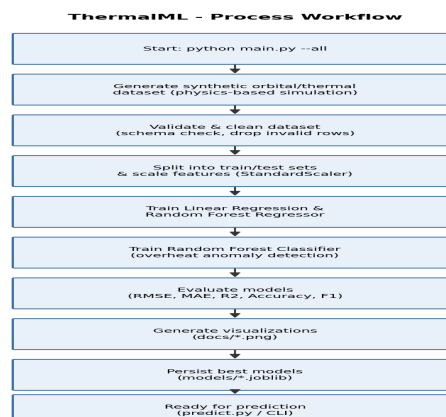


Figure 2: End-to-end process workflow

6. Design Diagrams

6.1 UML Class Diagram

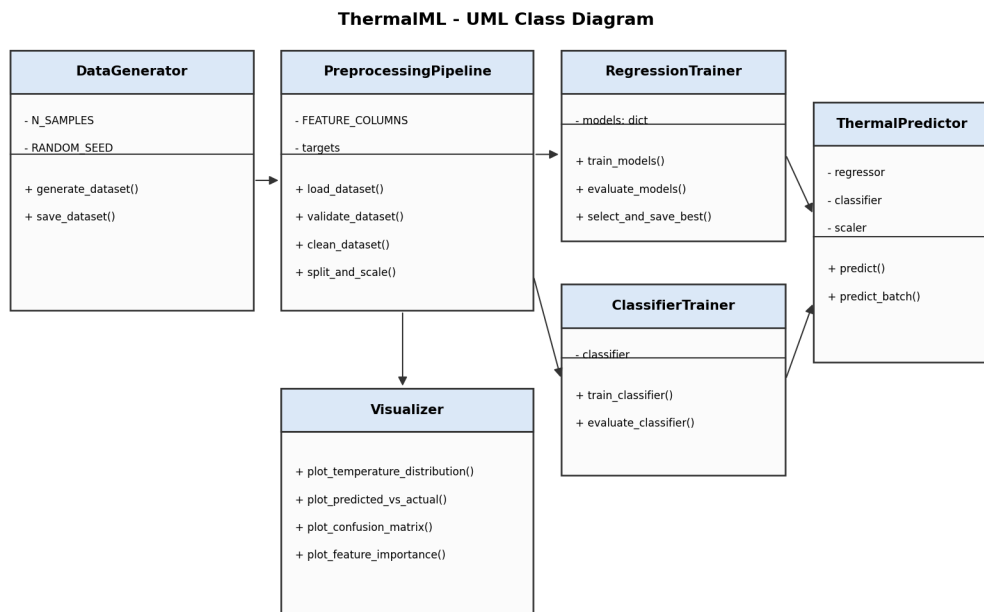


Figure 3: UML class diagram

6.2 Sequence Diagram

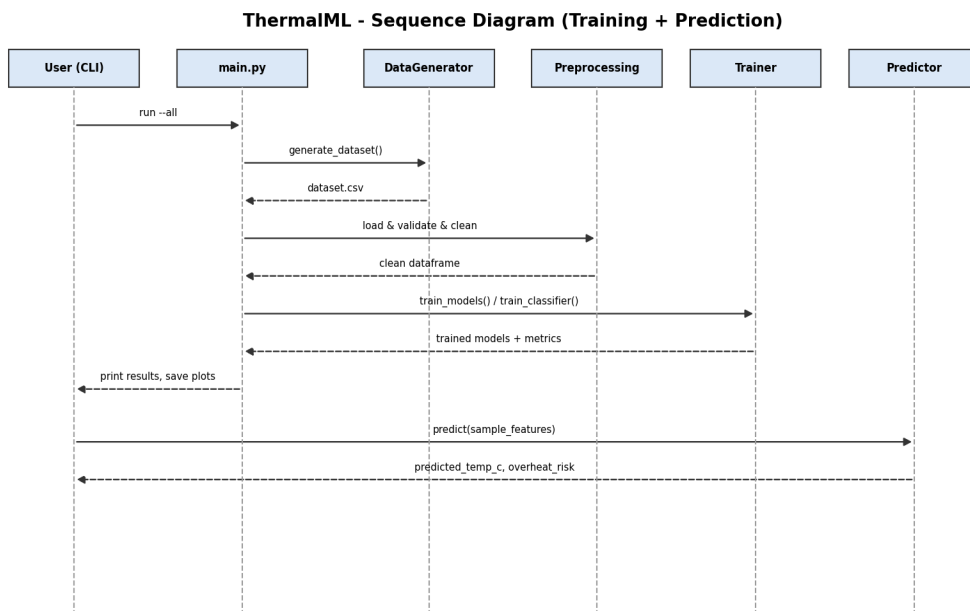


Figure 4: Sequence diagram — training and prediction flow

6.3 Database / Storage Design

This project does not use a relational database. Data is persisted as flat files: the generated dataset as CSV (`data/thermal_dataset.csv`) and trained models as serialized joblib objects (`models/*.joblib`). This

is appropriate given the project's scope — a single tabular dataset with no relational structure or concurrent-write requirements — so an ER diagram/schema is not applicable.

7. Dataset Description

The dataset (5,000 rows) is synthetically generated using a first-order radiative energy balance equation: $Q_{\text{absorbed}} = \alpha \cdot A \cdot (G_{\text{solar}} \cdot \cos \theta + G_{\text{albedo}} + G_{\text{earth_ir}})$, and $Q_{\text{emitted}} = \epsilon \cdot \sigma \cdot A \cdot T^4$, solved for steady-state T . Ten input features are sampled from physically realistic ranges (altitude 300–1200 km, sun angle 0–120°, absorptivity/emissivity 0.2–0.95, etc.), producing a continuous temperature target (`component_temp_c`) and a derived binary `overheat_flag` (threshold: 85°C). Small Gaussian noise is added to emulate sensor/model uncertainty.

8. Model Selection Rationale

Regression: Linear Regression was chosen as a fast, interpretable baseline; Random Forest was chosen for its ability to capture the non-linear T^4 relationship inherent in the underlying physics without manual feature engineering. The model with the higher R^2 on the held-out test set is automatically selected and persisted.

Classification: Random Forest was used for overheat detection for the same non-linearity-capturing reason, with `class_weight='balanced'` to handle any class imbalance between normal and overheat-risk samples.

9. Evaluation Methodology

An 80/20 train-test split (fixed random seed for reproducibility) is used for both tasks. Regression is evaluated using RMSE, MAE, and R^2 . Classification is evaluated using Accuracy, Precision, Recall, F1-score, and a confusion matrix. Feature importance plots are generated for both the regressor and classifier to interpret which physical parameters drive predictions.

10. Implementation Details

The project is implemented in Python 3 using pandas/NumPy for data handling, scikit-learn for modeling, Matplotlib for visualization, and joblib for model persistence. The codebase is organized into 8 focused modules under `src/` plus a CLI entry point (`main.py`) and a test suite (`tests/`) — satisfying the modular, multi-file structure expected for a coding-based project. Configuration constants are centralized, and a shared logger provides consistent monitoring output across all stages.

11. Screenshots / Results

11.1 Dataset Characteristics

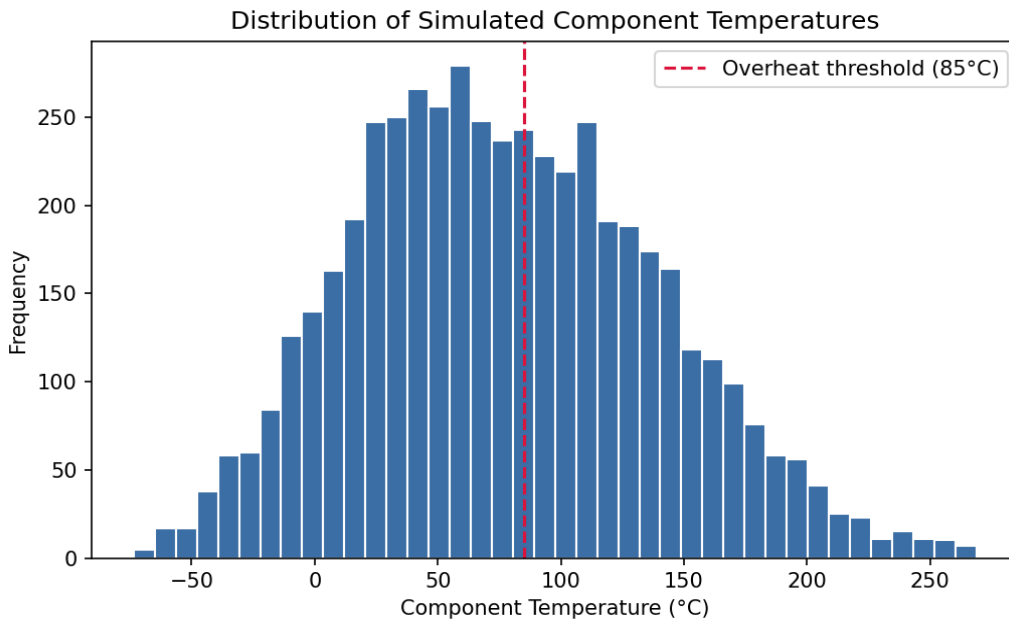


Figure 5: Distribution of simulated component temperatures

11.2 Regression Performance

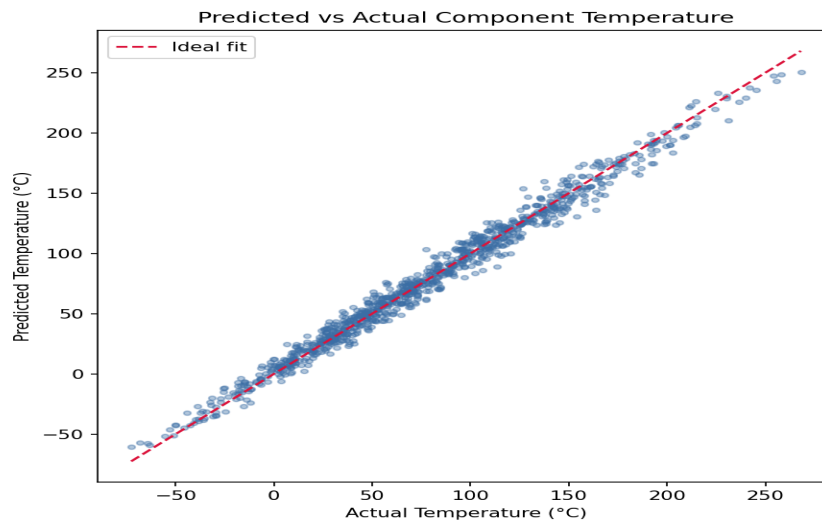


Figure 6: Predicted vs. actual temperature (Random Forest)

Model	RMSE (°C)	MAE (°C)	R2
Linear Regression	13.55	10.71	0.9499
Random Forest (selected)	7.54	5.91	0.9845

11.3 Classification Performance

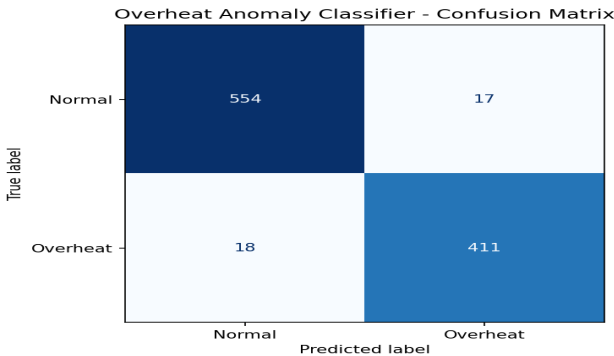


Figure 7: Overheat classifier confusion matrix

Metric	Value
Accuracy	0.9650
Precision	0.9603
Recall	0.9580
F1-score	0.9592

11.4 Feature Importance

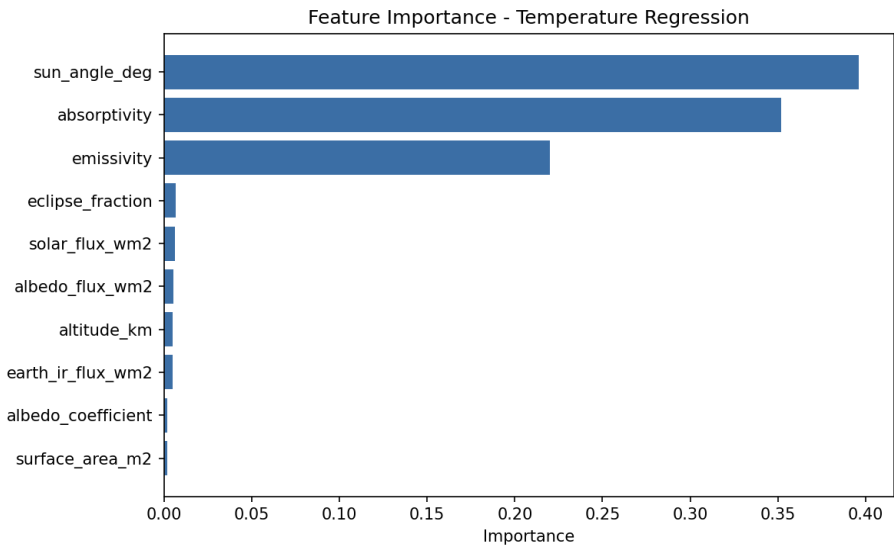


Figure 8: Feature importance — temperature regression

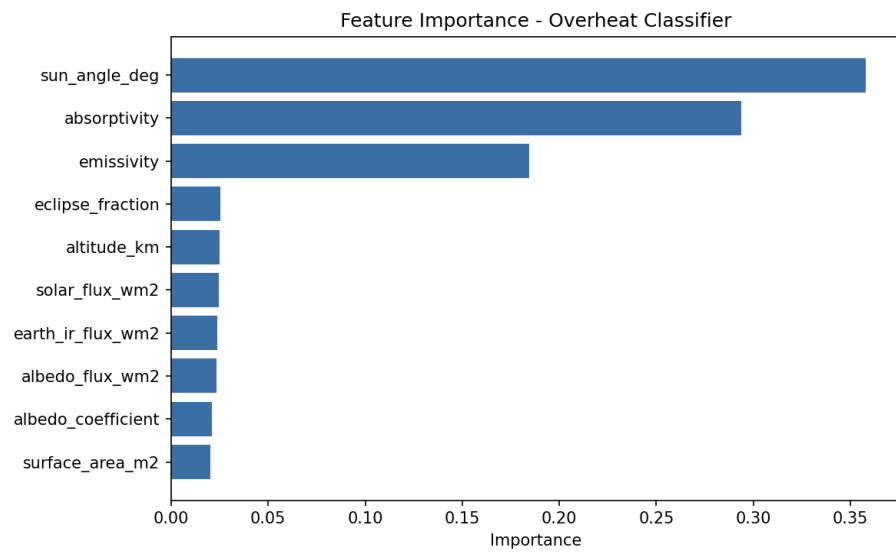


Figure 9: Feature importance — overheat classifier

12. Testing Approach

A unit test suite (tests/test_pipeline.py, 6 tests) validates: (1) generated dataset shape and required columns, (2) physical plausibility of generated temperatures (bounded to a generous realistic range), (3) internal consistency between temperature and the overheat flag, (4) successful validation of well-formed data, (5) correct rejection of malformed data (missing columns), and (6) correct removal of invalid rows during cleaning. All 6 tests pass (python -m unittest discover tests).

13. Challenges Faced

- No public dataset exists linking orbital parameters directly to component temperature, since real telemetry is proprietary — solved by deriving a synthetic dataset from the governing radiative energy balance equation rather than an ad-hoc random dataset.
- Balancing physical realism against reproducibility: fixed random seeds and clearly bounded parameter ranges were used so results are consistent across runs.
- Avoiding data leakage between the regression and classification tasks, since both share the same underlying features — addressed by fitting the feature scaler once during the regression stage and reusing it (not refitting) for the classification stage.

14. Learnings & Key Takeaways

- Random Forest models handle the inherent non-linearity of the T^4 radiative relationship far better than linear models, without needing manual polynomial feature engineering.
- Physics-based synthetic data generation is a practical way to build a defensible, reproducible dataset when real-world data is inaccessible.
- A clean modular pipeline (separate data, training, evaluation, and prediction modules) makes it straightforward to swap in different models or extend to new features.

15. Future Enhancements

- Extend to a transient (time-dependent) thermal model instead of a steady-state approximation, to capture temperature evolution across an orbit.
- Incorporate real telemetry data (where available) to validate and fine-tune the synthetic-data-trained models.
- Add a lightweight web dashboard for interactive predictions instead of the current CLI.
- Experiment with gradient boosting models (XGBoost/LightGBM) for further accuracy gains.

16. References

[1] Gilmore, D. G. (ed.), *Spacecraft Thermal Control Handbook*, Vol. I: Fundamental Technologies, The Aerospace Press, 2002.

[2] Pedregosa, F. et al., "Scikit-learn: Machine Learning in Python," *Journal of Machine Learning Research*, 12, 2011.

[3] NASA, "Basics of Space Flight — Thermal Control," NASA JPL, jpl.nasa.gov.